

# METİN DÜZEYİNDE BİT TEMSİLİ

- Karakter Kodlama ve İkili Sistem ?
- Bilgisayarlar yalnızca elektrik sinyalleriyle, yani **0 ve 1'lerden (Bitler)** oluşan ikili kodlarla çalışır. Bizim kullandığımız harflerin, rakamların ve sembollerin bu 0 ve 1'lere dönüştürülmesi gerekir. Bu dönüştürme işlemine **Karakter Kodlama (Character Encoding)** adı verilir.
- 1960'lı yıllara kadar her bilgisayar üreticisi bu eşleştirmeyi farklı yaptığı için, bir metin bir bilgisayardan diğerine güvenilir şekilde aktarılamıyordu. Bu karmaşayı çözmek için bir **standartlaşmaya** ihtiyaç duyuldu.

| Dec | Hx | Oct | Char                        | Dec | Hx | Oct | Html        | Chr | Dec | Hx | Oct | Html    | Chr | Dec |
|-----|----|-----|-----------------------------|-----|----|-----|-------------|-----|-----|----|-----|---------|-----|-----|
| 0   | 0  | 000 | NUL (null)                  | 32  | 20 | 040 | &#32; Space |     | 64  | 40 | 100 | &#64; @ |     | 96  |
| 1   | 1  | 001 | SOH (start of heading)      | 33  | 21 | 041 | &#33; !     |     | 65  | 41 | 101 | &#65; A |     | 97  |
| 2   | 2  | 002 | STX (start of text)         | 34  | 22 | 042 | &#34; "     |     | 66  | 42 | 102 | &#66; B |     | 98  |
| 3   | 3  | 003 | ETX (end of text)           | 35  | 23 | 043 | &#35; #     |     | 67  | 43 | 103 | &#67; C |     | 99  |
| 4   | 4  | 004 | EOT (end of transmission)   | 36  | 24 | 044 | &#36; \$    |     | 68  | 44 | 104 | &#68; D |     | 100 |
| 5   | 5  | 005 | ENQ (enquiry)               | 37  | 25 | 045 | &#37; %     |     | 69  | 45 | 105 | &#69; E |     | 101 |
| 6   | 6  | 006 | ACK (acknowledge)           | 38  | 26 | 046 | &#38; &     |     | 70  | 46 | 106 | &#70; F |     | 102 |
| 7   | 7  | 007 | BEL (bell)                  | 39  | 27 | 047 | &#39; '     |     | 71  | 47 | 107 | &#71; G |     | 103 |
| 8   | 8  | 010 | BS (backspace)              | 40  | 28 | 050 | &#40; (     |     | 72  | 48 | 110 | &#72; H |     | 104 |
| 9   | 9  | 011 | TAB (horizontal tab)        | 41  | 29 | 051 | &#41; )     |     | 73  | 49 | 111 | &#73; I |     | 105 |
| 10  | A  | 012 | LF (NL line feed, new line) | 42  | 2A | 052 | &#42; *     |     | 74  | 4A | 112 | &#74; J |     | 106 |
| 11  | B  | 013 | VT (vertical tab)           | 43  | 2B | 053 | &#43; +     |     | 75  | 4B | 113 | &#75; K |     | 107 |
| 12  | C  | 014 | FF (NP form feed, new page) | 44  | 2C | 054 | &#44; ,     |     | 76  | 4C | 114 | &#76; L |     | 108 |
| 13  | D  | 015 | CR (carriage return)        | 45  | 2D | 055 | &#45; -     |     | 77  | 4D | 115 | &#77; M |     | 109 |
| 14  | E  | 016 | SO (shift out)              | 46  | 2E | 056 | &#46; .     |     | 78  | 4E | 116 | &#78; N |     | 110 |
| 15  | F  | 017 | SI (shift in)               | 47  | 2F | 057 | &#47; /     |     | 79  | 4F | 117 | &#79; O |     | 111 |
| 16  | 10 | 020 | DLE (data link escape)      | 48  | 30 | 060 | &#48; 0     |     | 80  | 50 | 120 | &#80; P |     | 112 |
| 17  | 11 | 021 | DC1 (device control 1)      | 49  | 31 | 061 | &#49; 1     |     | 81  | 51 | 121 | &#81; Q |     | 113 |
| 18  | 12 | 022 | DC2 (device control 2)      | 50  | 32 | 062 | &#50; 2     |     | 82  | 52 | 122 | &#82; R |     | 114 |
| 19  | 13 | 023 | DC3 (device control 3)      | 51  | 33 | 063 | &#51; 3     |     | 83  | 53 | 123 | &#83; S |     | 115 |
| 20  | 14 | 024 | DC4 (device control 4)      | 52  | 34 | 064 | &#52; 4     |     | 84  | 54 | 124 | &#84; T |     | 116 |
| 21  | 15 | 025 | NAK (negative acknowledge)  | 53  | 35 | 065 | &#53; 5     |     | 85  | 55 | 125 | &#85; U |     | 117 |
| 22  | 16 | 026 | SYN (synchronous idle)      | 54  | 36 | 066 | &#54; 6     |     | 86  | 56 | 126 | &#86; V |     | 118 |
| 23  | 17 | 027 | ETB (end of trans. block)   | 55  | 37 | 067 | &#55; 7     |     | 87  | 57 | 127 | &#87; W |     | 119 |
| 24  | 18 | 030 | CAN (cancel)                | 56  | 38 | 070 | &#56; 8     |     | 88  | 58 | 130 | &#88; X |     | 120 |
| 25  | 19 | 031 | EM (end of medium)          | 57  | 39 | 071 | &#57; 9     |     | 89  | 59 | 131 | &#89; Y |     | 121 |
| 26  | 1A | 032 | SUB (substitute)            | 58  | 3A | 072 | &#58; :     |     | 90  | 5A | 132 | &#90; Z |     | 122 |
| 27  | 1B | 033 | ESC (escape)                | 59  | 3B | 073 | &#59; ;     |     | 91  | 5B | 133 | &#91; [ |     | 123 |
| 28  | 1C | 034 | FS (file separator)         | 60  | 3C | 074 | &#60; <     |     | 92  | 5C | 134 | &#92; \ |     | 124 |
| 29  | 1D | 035 | GS (group separator)        | 61  | 3D | 075 | &#61; =     |     | 93  | 5D | 135 | &#93; ] |     | 125 |
| 30  | 1E | 036 | RS (record separator)       | 62  | 3E | 076 | &#62; >     |     | 94  | 5E | 136 | &#94; ^ |     | 126 |
| 31  | 1F | 037 | US (unit separator)         | 63  | 3F | 077 | &#63; ?     |     | 95  | 5F | 137 | &#95; _ |     | 127 |

---

# ASCII STANDARDI

- Bu standartlaşma ihtiyacına yanıt olarak **ASCII (American Standard Code for Information Interchange)** geliştirildi.
  - **Yapı:** ASCII, her karakter için **7 bit** kullanır. Bu sayede toplam **128 farklı karakter** kodlanabilir.
  - **Kapsam:** İngiliz alfabesi (küçük/büyük harf), rakamlar, temel noktalama işaretleri ve metnin düzenini sağlayan **kontrol karakterlerini** içerir.
-

# ASCII'NİN SINIRLAMALARI

- Adından da anlaşılacağı gibi ASCII, **Amerikan standardı** temel alınarak hazırlandığı için ciddi sınırlamalara sahipti:
- Dil Sorunu:** Türkçe (ş, ç, ğ, ö, ü) gibi özel karakterleri veya diğer dünya dillerindeki karakterleri kodlayamıyordu. Bir programda Türkçe karakter kullanmak hataya neden oluyordu.
- Kullanım:** ASCII, basit sistemlerde (kullanıcı adı/parola gibi) hala temel olarak kullanılsa da, küresel metinler için yetersizdir.

## ASCII TABLE

| Decimal | Hexadecimal | Binary | Octal | Char                   | Decimal | Hexadecimal | Binary  | Octal | Char | Decimal | Hexadecimal | Binary  | Octal | Char  |
|---------|-------------|--------|-------|------------------------|---------|-------------|---------|-------|------|---------|-------------|---------|-------|-------|
| 0       | 0           | 0      | 0     | [NULL]                 | 48      | 30          | 110000  | 60    | 0    | 96      | 60          | 1100000 | 140   | .     |
| 1       | 1           | 1      | 1     | [START OF HEADING]     | 49      | 31          | 110001  | 61    | 1    | 97      | 61          | 1100001 | 141   | a     |
| 2       | 2           | 10     | 2     | [START OF TEXT]        | 50      | 32          | 110010  | 62    | 2    | 98      | 62          | 1100010 | 142   | b     |
| 3       | 3           | 11     | 3     | [END OF TEXT]          | 51      | 33          | 110011  | 63    | 3    | 99      | 63          | 1100011 | 143   | c     |
| 4       | 4           | 100    | 4     | [END OF TRANSMISSION]  | 52      | 34          | 110100  | 64    | 4    | 100     | 64          | 1100100 | 144   | d     |
| 5       | 5           | 101    | 5     | [ENQUIRY]              | 53      | 35          | 110101  | 65    | 5    | 101     | 65          | 1100101 | 145   | e     |
| 6       | 6           | 110    | 6     | [ACKNOWLEDGE]          | 54      | 36          | 110110  | 66    | 6    | 102     | 66          | 1100110 | 146   | f     |
| 7       | 7           | 111    | 7     | [BELL]                 | 55      | 37          | 110111  | 67    | 7    | 103     | 67          | 1100111 | 147   | g     |
| 8       | 8           | 1000   | 10    | [BACKSPACE]            | 56      | 38          | 111000  | 70    | 8    | 104     | 68          | 1101000 | 150   | h     |
| 9       | 9           | 1001   | 11    | [HORIZONTAL TAB]       | 57      | 39          | 111001  | 71    | 9    | 105     | 69          | 1101001 | 151   | i     |
| 10      | A           | 1010   | 12    | [LINE FEED]            | 58      | 3A          | 111010  | 72    | :    | 106     | 6A          | 1101010 | 152   | j     |
| 11      | B           | 1011   | 13    | [VERTICAL TAB]         | 59      | 3B          | 111011  | 73    | ;    | 107     | 6B          | 1101011 | 153   | k     |
| 12      | C           | 1100   | 14    | [FORM FEED]            | 60      | 3C          | 111100  | 74    | <    | 108     | 6C          | 1101100 | 154   | l     |
| 13      | D           | 1101   | 15    | [CARRIAGE RETURN]      | 61      | 3D          | 111101  | 75    | =    | 109     | 6D          | 1101101 | 155   | m     |
| 14      | E           | 1110   | 16    | [SHIFT OUT]            | 62      | 3E          | 111110  | 76    | >    | 110     | 6E          | 1101110 | 156   | n     |
| 15      | F           | 1111   | 17    | [SHIFT IN]             | 63      | 3F          | 111111  | 77    | ?    | 111     | 6F          | 1101111 | 157   | o     |
| 16      | 10          | 10000  | 20    | [DATA LINK ESCAPE]     | 64      | 40          | 1000000 | 100   | @    | 112     | 70          | 1110000 | 160   | p     |
| 17      | 11          | 10001  | 21    | [DEVICE CONTROL 1]     | 65      | 41          | 1000001 | 101   | A    | 113     | 71          | 1110001 | 161   | q     |
| 18      | 12          | 10010  | 22    | [DEVICE CONTROL 2]     | 66      | 42          | 1000010 | 102   | B    | 114     | 72          | 1110010 | 162   | r     |
| 19      | 13          | 10011  | 23    | [DEVICE CONTROL 3]     | 67      | 43          | 1000011 | 103   | C    | 115     | 73          | 1110011 | 163   | s     |
| 20      | 14          | 10100  | 24    | [DEVICE CONTROL 4]     | 68      | 44          | 1000100 | 104   | D    | 116     | 74          | 1110100 | 164   | t     |
| 21      | 15          | 10101  | 25    | [NEGATIVE ACKNOWLEDGE] | 69      | 45          | 1000101 | 105   | E    | 117     | 75          | 1110101 | 165   | u     |
| 22      | 16          | 10110  | 26    | [SYNCHRONOUS IDLE]     | 70      | 46          | 1000110 | 106   | F    | 118     | 76          | 1110110 | 166   | v     |
| 23      | 17          | 10111  | 27    | [ENG OF TRANS. BLOCK]  | 71      | 47          | 1000111 | 107   | G    | 119     | 77          | 1110111 | 167   | w     |
| 24      | 18          | 11000  | 30    | [CANCEL]               | 72      | 48          | 1001000 | 110   | H    | 120     | 78          | 1111000 | 170   | x     |
| 25      | 19          | 11001  | 31    | [END OF MEDIUM]        | 73      | 49          | 1001001 | 111   | I    | 121     | 79          | 1111001 | 171   | y     |
| 26      | 1A          | 11010  | 32    | [SUBSTITUTE]           | 74      | 4A          | 1001010 | 112   | J    | 122     | 7A          | 1111010 | 172   | z     |
| 27      | 1B          | 11011  | 33    | [ESCAPE]               | 75      | 4B          | 1001011 | 113   | K    | 123     | 7B          | 1111011 | 173   | {     |
| 28      | 1C          | 11100  | 34    | [FILE SEPARATOR]       | 76      | 4C          | 1001100 | 114   | L    | 124     | 7C          | 1111100 | 174   |       |
| 29      | 1D          | 11101  | 35    | [GROUP SEPARATOR]      | 77      | 4D          | 1001101 | 115   | M    | 125     | 7D          | 1111101 | 175   | }     |
| 30      | 1E          | 11110  | 36    | [RECORD SEPARATOR]     | 78      | 4E          | 1001110 | 116   | N    | 126     | 7E          | 1111110 | 176   | ~     |
| 31      | 1F          | 11111  | 37    | [UNIT SEPARATOR]       | 79      | 4F          | 1001111 | 117   | O    | 127     | 7F          | 1111111 | 177   | [DEL] |
| 32      | 20          | 100000 | 40    | [SPACE]                | 80      | 50          | 1010000 | 120   | P    |         |             |         |       |       |
| 33      | 21          | 100001 | 41    | !                      | 81      | 51          | 1010001 | 121   | Q    |         |             |         |       |       |
| 34      | 22          | 100010 | 42    | "                      | 82      | 52          | 1010010 | 122   | R    |         |             |         |       |       |
| 35      | 23          | 100011 | 43    | #                      | 83      | 53          | 1010011 | 123   | S    |         |             |         |       |       |
| 36      | 24          | 100100 | 44    | \$                     | 84      | 54          | 1010100 | 124   | T    |         |             |         |       |       |
| 37      | 25          | 100101 | 45    | %                      | 85      | 55          | 1010101 | 125   | U    |         |             |         |       |       |
| 38      | 26          | 100110 | 46    | &                      | 86      | 56          | 1010110 | 126   | V    |         |             |         |       |       |
| 39      | 27          | 100111 | 47    | '                      | 87      | 57          | 1010111 | 127   | W    |         |             |         |       |       |
| 40      | 28          | 101000 | 50    | (                      | 88      | 58          | 1011000 | 130   | X    |         |             |         |       |       |
| 41      | 29          | 101001 | 51    | )                      | 89      | 59          | 1011001 | 131   | Y    |         |             |         |       |       |
| 42      | 2A          | 101010 | 52    | *                      | 90      | 5A          | 1011010 | 132   | Z    |         |             |         |       |       |
| 43      | 2B          | 101011 | 53    | +                      | 91      | 5B          | 1011011 | 133   | [    |         |             |         |       |       |
| 44      | 2C          | 101100 | 54    | ,                      | 92      | 5C          | 1011100 | 134   | \    |         |             |         |       |       |
| 45      | 2D          | 101101 | 55    | -                      | 93      | 5D          | 1011101 | 135   | ]    |         |             |         |       |       |
| 46      | 2E          | 101110 | 56    | .                      | 94      | 5E          | 1011110 | 136   | ^    |         |             |         |       |       |
| 47      | 2F          | 101111 | 57    | /                      | 95      | 5F          | 1011111 | 137   | _    |         |             |         |       |       |

---

- **Genişletilmiş ASCII (Geçici Çözüm)**

- Sonraki yıllarda bilgisayarların 8. biti de kullanmasıyla (toplam 256 karakter), bu boşluk farklı dillerdeki karakterleri (Türkçe dahil) eklemek için kullanıldı ve buna **Genişletilmiş ASCII** dendi. Ancak bu çözüm de evrensel değildi, çünkü farklı bölgeler 8. biti farklı karakterler için kullandı, bu da karmaşayı tam olarak çözemedi.

# NİHAİ ÇÖZÜM – UNICODE

- Unicode, dünyadaki **her karaktere** (harf, sayı, emoji) benzersiz ve tek bir numara atayan **evrensel bir standarttır**.
- Bu, dil veya platform fark etmeksizin, metinlerin **her yerde aynı** şekilde görünmesini garanti eder.
- Unicode, karakterlerin sadece **numarasını** belirler; bilgisayarda nasıl saklanacağını ise **UTF** sistemleri belirler.

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |                                    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|------------------------------------|
| 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F | Latince yazı                       |
| 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 1A | 1B | 1C | 1D | 1E | 1F | Latin olmayan Avrupa yazıları      |
| 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 2A | 2B | 2C | 2D | 2E | 2F | Afrika yazıları                    |
| 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 3A | 3B | 3C | 3D | 3E | 3F | Orta Doğu ve Güneybatı Asya yazını |
| 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 4A | 4B | 4C | 4D | 4E | 4F | Güney ve Orta Asya yazıları        |
| 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 5A | 5B | 5C | 5D | 5E | 5F | Güneydoğu Asya yazını              |
| 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 6A | 6B | 6C | 6D | 6E | 6F | Doğu Asya yazıları                 |
| 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 7A | 7B | 7C | 7D | 7E | 7F | Çince ve Japonca karakterler       |
| 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 8A | 8B | 8C | 8D | 8E | 8F | Endonezya ve Okyanusya yazıları    |
| 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 9A | 9B | 9C | 9D | 9E | 9F | Amerikan yazını                    |
| A0 | A1 | A2 | A3 | A4 | A5 | A6 | A7 | A8 | A9 | AA | AB | AC | AD | AE | AF | Notasyonel sistemler               |
| B0 | B1 | B2 | B3 | B4 | B5 | B6 | B7 | B8 | B9 | BA | BB | BC | BD | BE | BF | Semboller                          |
| C0 | C1 | C2 | C3 | C4 | C5 | C6 | C7 | C8 | C9 | CA | CB | CC | CD | CE | CF | Özel kullanım                      |
| D0 | D1 | D2 | D3 | D4 | D5 | D6 | D7 | D8 | D9 | DA | DB | DC | DD | DE | DF | UTF-16 vekilleri                   |
| E0 | E1 | E2 | E3 | E4 | E5 | E6 | E7 | E8 | E9 | EA | EB | EC | ED | EE | EF | Ayrılmamış kod noktaları           |
| F0 | F1 | F2 | F3 | F4 | F5 | F6 | F7 | F8 | F9 | FA | FB | FC | FD | FE | FF | Unicode 17.0'den itibaren          |

---

- **Depolama Biçimi – UTF-8**

- **UTF-8** (Unicode Transformation Format) bu numaraları bilgisayarın belleğinde saklamak için en yaygın kullanılan formattır.
- En önemli özelliği **Değişken Uzunluklu** olmasıdır.
- İngilizce harfler (ASCII uyumluluğu için) sadece **1 bayt** kullanır. (Yer tasarrufu)
- Türkçe, Arapça, Çince gibi dillerdeki özel karakterler **2, 3 veya 4 bayt** kullanır.
- Bu esneklik ve evrensellik sayesinde UTF-8, **internetin ve modern sistemlerin standardı** haline gelmiştir.

Code point ↔ UTF-8 conversion

| First code point | Last code point | Byte 1    | Byte 2   | Byte 3   | Byte 4   |
|------------------|-----------------|-----------|----------|----------|----------|
| U+0000           | U+007F          | 0yyyyzzzz |          |          |          |
| U+0080           | U+07FF          | 110xxxxyy | 10yyzzzz |          |          |
| U+0800           | U+FFFF          | 1110www   | 10xxxxyy | 10yyzzzz |          |
| U+10000          | U+10FFFF        | 11110uvv  | 10vvwww  | 10xxxxyy | 10yyzzzz |



---

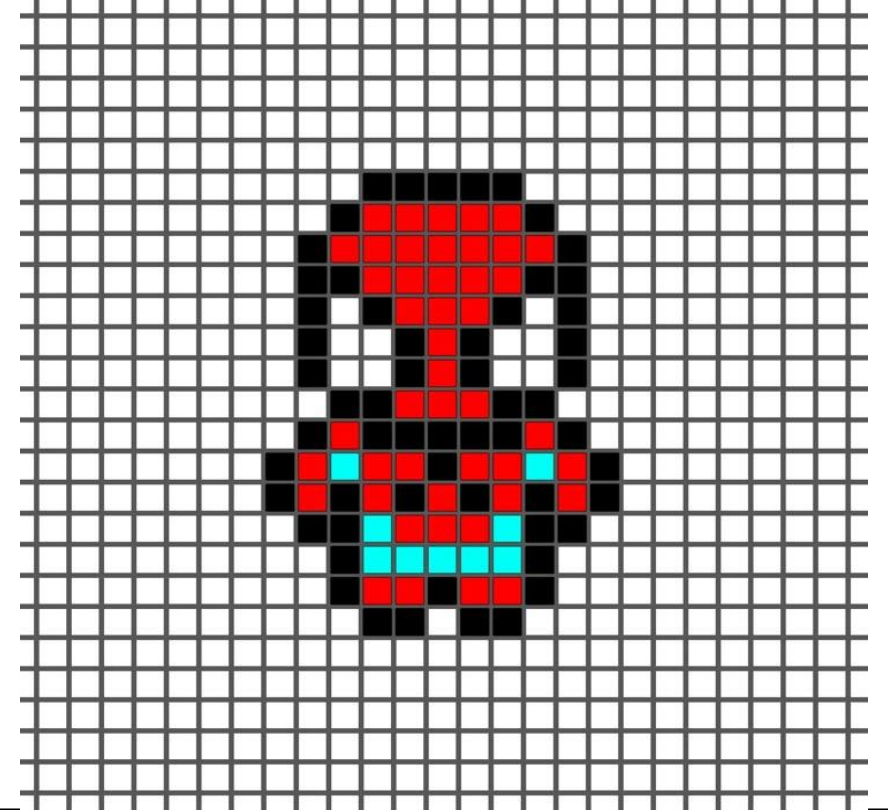
# RESİM DÜZEYİNDE BİT TEMSİLİ

- **Resimler Neden Bitlere Dönüşür?**
  - Tüm dijital veriler gibi, resimler de bilgisayarda **0 ve 1'lerden** (Bitler) oluşur.
  - Bir resim, milyonlarca küçük kareden oluşan bir **ızgaradır (Raster Grafik)**.
  - Bu küçük karelerin her biri, resmin en temel yapı taşıdır: **Piksel**.
-

---

# PİKSEL NEDİR?

- **Piksel:** Bir resimdeki tek bir rengi taşıyan en küçük birimdir.
- **Çözünürlük:** Bir resmin toplam piksel sayısını belirtir (Örn: 1920 x 1080 piksel).
- Çözünürlük ne kadar yüksekse, resim o kadar fazla piksel, dolayısıyla o kadar fazla **bit bilgisi** içerir ve dosya boyutu büyür.





---

- **Renk Nasıl Kodlanır? (Bit Derinliği)**

- **Bit Derinliği:** Her bir pikselin rengini depolamak için kullanılan bit sayısıdır. Bu, bir pikselin alabileceği toplam renk sayısını belirler.

- **Örnekler:**

- **1 Bit:** Sadece 2 renk (Siyah/Beyaz).

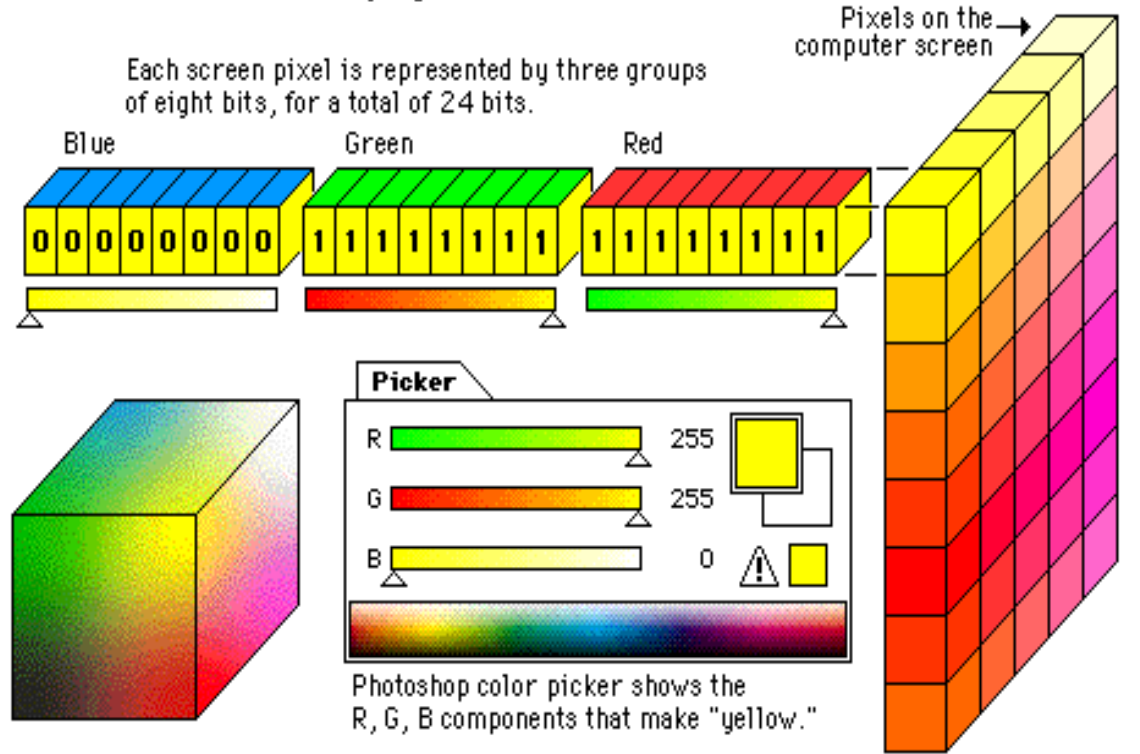
- **8 Bit:** 256 renk.

- **24 Bit (Gerçek Renk):** 16.7 Milyon renk. (En yaygın ve doğal renkler için kullanılan standarttır.)

# 24 Bit ve RGB Sistemi

- Bilgisayarlar renkleri genellikle **RGB (Kırmızı, Yeşil, Mavi)** sistemiyle kodlar.
- 24 Bit derinlikte: Renk bilgisi, eşit olarak 3 ana bileşene ayrılır.
- Her bir ana renk (Kırmızı, Yeşil, Mavi) için **8 bit** ayrılır. ( $8 + 8 + 8 = 24$ )
- 8 bit, her ana rengin **256 farklı yoğunluğunu** kodlamayı sağlar (0'dan 255'e kadar).

## 24-bit "true color" displays



---

## • Dosya Boyutu Yönetimi ve Sıkıştırma Yöntemleri

### Bit Yükünü Azaltmak

- **Ham Bit Boyutu:** {Piksel Sayısı} x {Bit Derinliği} formülüyle hesaplanır. Yüksek çözünürlüklü resimler, büyük bit hacmine sahiptir.
- Bu bit yükünü azaltmak için **Sıkıştırma** yöntemleri kullanılır.

### Kayıpsız Sıkıştırma (Lossless)

**Amaç:** Resmin tüm bitlerini **koruyarak** boyutu küçültmek. Orijinal kalitede veri kaybı olmaz.

**Format Örneği: PNG (Portable Network Graphics)**

**Bit Kullanımı:** Bitleri silmez; tekrarlanan bit dizilerini matematiksel kodlarla temsil eder.

#### **Kullanım Alanları:**

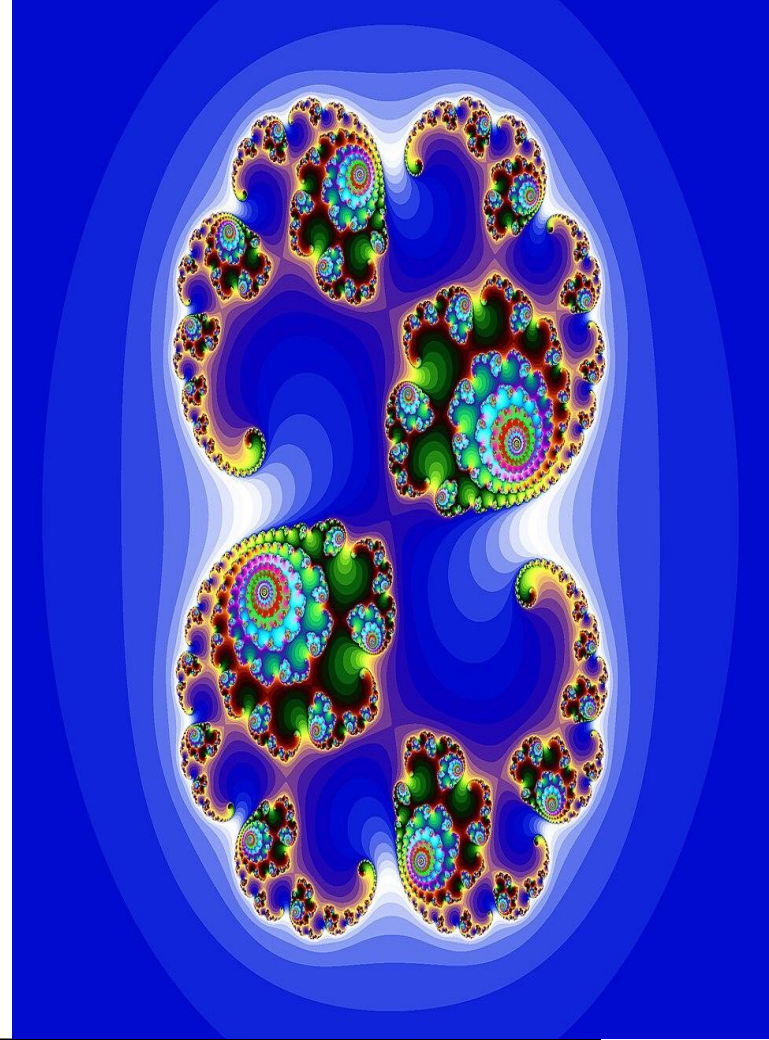
Logolar, ikonlar ve infografikler.

Saydam (şeffaf) arka plan gerektiren her türlü web grafiği.

Ekran görüntüleri ve yazılım arayüzleri.



- 
- **Kayıplı Sıkıştırma (Lossy)**
  - **Amaç:** Gözün algılamayacağı detaylara ait bitleri **kalıcı olarak silerek** dosya boyutunu maksimum küçültmek.
  - **Format Örneği: JPEG/JPG (Joint Photographic Experts Group)**
    - **Bit Kullanımı:** Bitler silindiği için dosya boyutu ciddi düşer, ancak kalite kaybedilir ve bloklaşma (artifaktlar) oluşabilir.
    - **Kullanım Alanları:**
      - Yüksek çözünürlüklü dijital **fotoğraflar** (manzara, portre vb.).
      - Sosyal medya paylaşımları ve web fotoğraf galerileri.
      - Renk geçişleri yumuşak olan, detay kaybının tolere edilebildiği durumlar.
- 



- 
- **Başka Bir Kayıpsız Format: GIF (Graphics Interchange Format)**
  - **Amaç:** Kısa animasyonlar oluşturmak ve düşük renkli grafikleri sıkıştırmak.
  - **Bit Kullanımı:** Kayıpsız sıkıştırma kullanır, ancak çok büyük bir kısıtlaması vardır: Maksimum **8 bit** derinlik (sadece 256 renk) destekler.
  - **Kullanım Alanları:**
    - Basit hareketli grafikler ve küçük animasyonlar (eğlenceli videolar).
    - Web sitelerinde kullanılan, 256'dan az renk içeren grafikler ve butonlar.
    - Fotoğraf kalitesine **uygun değildir** çünkü renk sayısı yetersiz kalır.



---

# SES VERİLERİNİN BİT DÜZEYİNDE TEMSİLİ

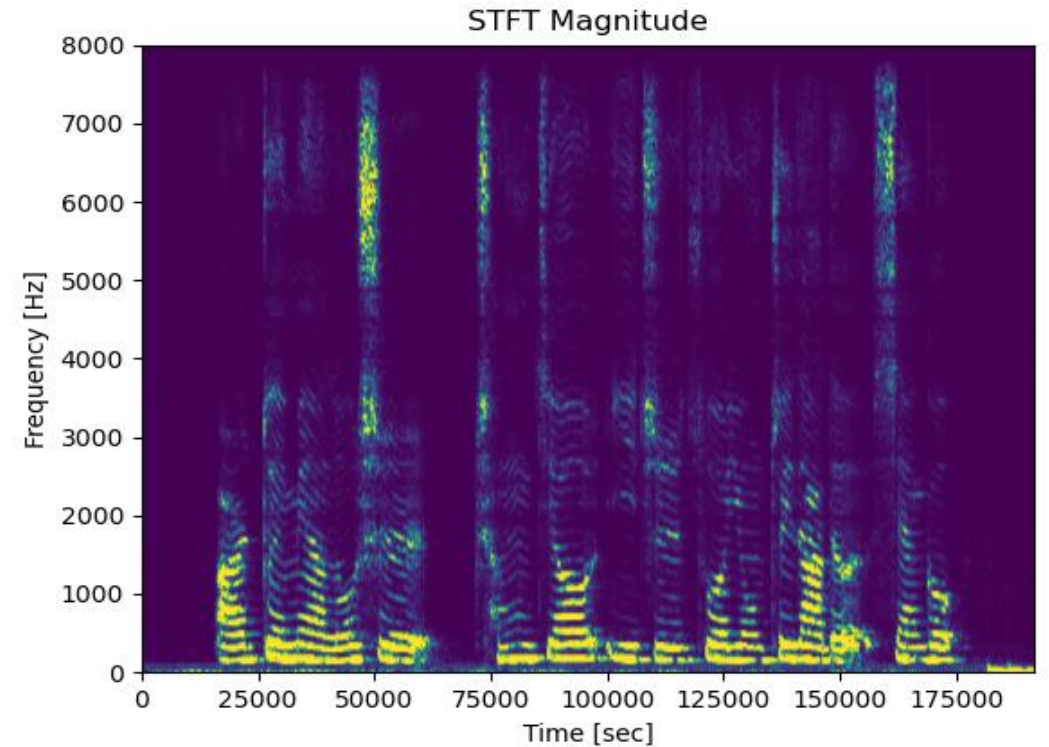
- **Sesin Dijitalleştirilmesi**

- Ses, doğası gereği **analog** ve sürekli bir dalgadır (Tıpkı su dalgaları gibi).
  - Bilgisayarlar yalnızca **dijital (kesikli 0 ve 1)** verileri işlediği için, ses dalgasının sürekli bilgisinin bitlere dönüştürülmesi gerekir.
  - Bu dönüştürme süreci iki ana adımdan oluşur: **Örnekleme (Sampling)** ve **Nicemleme (Quantization)**.
-



# ÖRNEKLEME (SAMPLING)

- Örnekleme:** Sürekli ses dalgasından belirli zaman aralıklarında **anlık değerler** (örnekler) almaktır.
- Örnekleme Hızı (Sample Rate):** Bir saniyede kaç tane anlık değer alındığını (okunduğunu) gösterir (Birim: Hertz veya kHz).
- Örnekleme hızı ne kadar yüksek olursa, ses dalgası o kadar **doğru** temsil edilir.
- Standart Örnekleme Hızı:** Müzik CD'leri için **44 kHz** kullanılır (Saniyede 44.100 örnek).





---

## • Nicemleme (Quantization)

- **Nicemleme:** Örnekleme sırasında alınan her bir anlık değerin (genlik/yükseklik) bir **sayısal karşılığa** (bit koduna) dönüştürülmesidir.
- **Bit Derinliği (Bit Depth):** Her bir örnek değeri temsil etmek için kullanılan bit sayısıdır. Bu, sesin hassasiyetini (dinamik aralığını) belirler.
- 16 Bit (CD Kalitesi) 65,536 olası genlik seviyesi.
- 24 Bit (Stüdyo Kalitesi) 16.7 milyon olası seviye.
- Bit derinliği ne kadar fazlaysa, ses o kadar net olur ve gürültü (hata) o kadar az olur.

## • Dijital Ses Dosyası Boyutu

- Dijital ses dosyasının boyutu (Bit sayısı), bu üç faktöre bağlıdır:
  - $\{\text{Dosya Boyutu}\} = \{\text{Örnekleme Hızı}\} \times \{\text{Bit Derinliği}\} \times \{\text{Süre}\} \times \{\text{Kanal Sayısı}\}$
  - **Kanal Sayısı:** Mono (1 kanal) veya Stereo (2 kanal).
  - **Örnek:** 1 dakikalık Stereo bir CD kalitesinde ses dosyası, yaklaşık 10 MB yer kaplar.
-

## • Ses Verisi Sıkıştırması

- Ses dosyaları genellikle çok büyük yer kapladığı için sıkıştırma yapılır.
- Kayıpsız Sıkıştırma (FLAC, WAV):** Orijinal bitlerin tamamını korur; kalite kaybı olmaz. Stüdyo veya arşivleme kalitesi için kullanılır.
- Kayıplı Sıkıştırma (MP3, AAC):** İnsan kulağının duymadığı veya ayırt edemediği frekanslardaki bitleri **silerek** boyutu küçültür.
- Kayıplı Sıkıştırma Mantığı:** Bitleri silerek dosya boyutunu %90'a kadar küçültebilir (Örn: MP3), ancak kalite kaybı olur.



---

# VERİ SIKIŞTIRMA NEDEN GEREKLİDİR?

- 1. Depolama Alanı Kısıtlamaları

- Dijital veriler (yüksek çözünürlüklü fotoğraflar, 4K videolar, stüdyo kalitesinde ses kayıtları) çok büyük bit hacmine sahiptir.
  - Sıkıştırma, aynı fiziksel alana (sabit disk, flash bellek) **daha fazla veri** sığdırmamızı sağlar.
  - **Örnek:** Bir fotoğrafı JPEG ile sıkıştırarak dosya boyutunu %90 oranında küçültebiliriz.
-

- **2. Daha Hızlı Aktarım (Bant Genişliği Tasarrufu)**
- İnternet ve ağ bağlantıları, saniyede aktarabileceği bit sayısı ile (bant genişliği) sınırlıdır.
- Sıkıştırılmış dosyalar, sıkıştırılmamış dosyalara göre çok daha **hızlı indirilir ve yüklenir**.
- **Örnek:** Web sayfalarının hızla açılması, büyük ölçüde resimlerin ve diğer dosyaların sıkıştırılmış olmasına bağlıdır.



---

### • 3. Maliyet Verimliliği

- Veri transfer maliyetini (özellikle mobil internet veya bulut depolama ücretlerini) düşürür.
- Bulut depolama sistemleri, daha az yer kaplayan sıkıştırılmış veriler sayesinde **daha az sunucu alanı** kullanır.

### 4. Teknoloji Uyum Zorunluluğu

.Bazı formatlar (Örn: MP3, MPEG video), dijital cihazlarda (telefon, müzik çalar) sorunsuz oynatılabilmek için standart bir boyutta olmalıdır. Sıkıştırma bu standardı sağlar.

---

---

# RUN-LENGTH ENCODING (RLE) GİBİ TEMEL SIKIŞTIRMA MANTIKLARI

## Run-Length Encoding (RLE)

- **İşlevi:** En temel sıkıştırma yöntemidir. **Ardışık Tekrar Sayımına** dayanır.
- **Nasıl Çalışır:** Yan yana gelen aynı değerlerin (piksel veya karakter) sayısını (uzunluğunu) tespit eder ve bu uzunluğu, değerin kendisiyle birlikte kodlar.
  - Örn: A A A B B B B B C C dizisi {3A 5B 2C} olarak kodlanır.
- **Kullanım Yeri:** Tek renkli alanların yoğun olduğu basit grafikler, ikonlar ve faks iletilen görüntüler.

- Example 1:

- Input: "AAAABBBCCDAA"

- Output: "4A3B2C1D2A"

- Example 2:

- Input:

"WWWWWWWWWWWWBWWWWWWWWWWWWBBBWWWWWWWWWWWWWWWWWWB"

- Output: "12WB12W3B24WB"

- Input: "AAAABBBCCDAA"

---



## • Huffman Kodlaması

•**İşlevi: Frekans Analizine** dayanan, kayıpsız sıkıştırmada en yaygın kullanılan temel algoritmalarından biridir.

•**Nasıl Çalışır:**

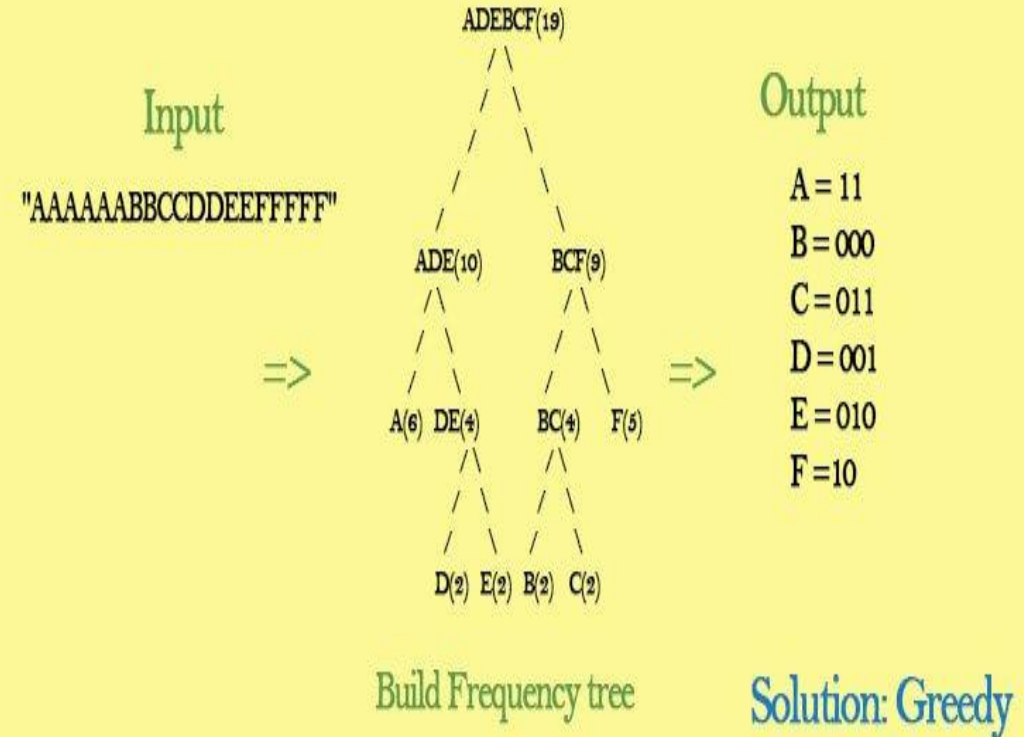
1. Veri akışındaki tüm karakterlerin **kullanım sıklığı (frekansı)** hesaplanır.

2. Karakterlere, frekanslarına göre **değişken uzunlukta** bit kodları atanır.

3. **Sık kullanılan** karakterlere **kısa kodlar**, az kullanılanlara ise uzun kodlar atanarak toplam bit sayısı azaltılır.

•**Kullanım Yeri:** Genel metin dosyaları, program kodları ve birçok modern dosya formatının (ZIP, GZIP, PNG) temel bir bileşeni olarak kullanılır.

## Huffman coding and decoding



---

- **LZW: Sözlük Tabanlı String Kodlama**

- **İşlevi:** Tekrarlayan kelimeleri veya karakter dizilerini (stringleri) bularak sıkıştırma yapan **Sözlük Tabanlı** bir yöntemdir.

- **Nasıl Çalışır:**

1. Veri okundukça, tekrar eden **uzun karakter dizileri** (örneğin "veri sıkıştırma") bir **sözlüğe** eklenir.

2. Bu uzun dizi, veri akışında tekrar görüldüğünde, kendisi yerine sözlükteki **daha kısa bir indeks koduyla** (örneğin 257) değiştirilir.

- **Kullanım Yeri:** Özellikle tekrarlayan veri kalıpları olan dosyalarda etkilidir. **GIF** görsel formatının ve bazı TIFF formatlarının temel sıkıştırma algoritmasıdır.

---

---

## • Aritmetik Kodlama: İstatistiksel Optimizasyon

• **İşlevi:** Huffman'dan daha gelişmiş bir **İstatistiksel Kodlama** yöntemidir.

• **Nasıl Çalışır:**

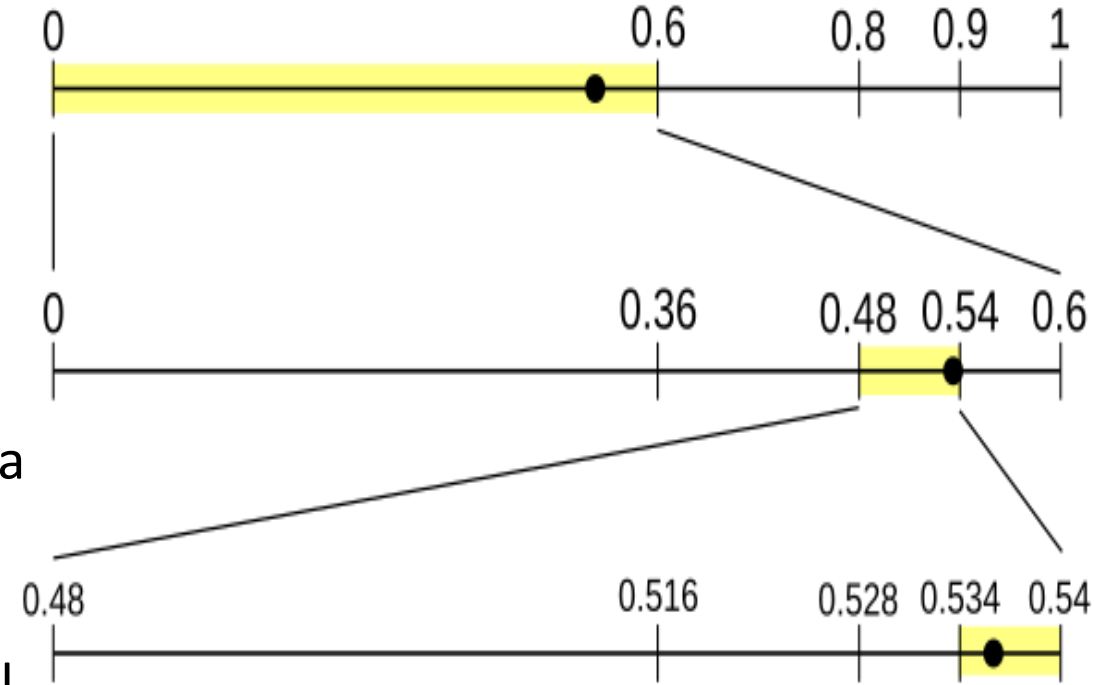
1. Tüm veri akışını 0 ile 1 arasında tek bir kesirli sayı aralığıyla temsil etmeye çalışır.

2. Her yeni sembol, bu aralığı kendi olasılığına göre daraltır.

3. Huffman'ın aksine, sembol başına tam sayı bit kullanma zorunluluğu yoktur. Bu sayede, sembollerin olasılıklarına daha yakın bir temsil sağlanır.

• **Kullanım Yeri:** Teorik olarak en yüksek sıkıştırma oranını sunar. JPEG 2000 gibi daha modern ve yüksek performans gerektiren standartlarda kullanılır.

---



---

- **Sonuç ve Ana Farklar**

- **RLE** basitliği ve hızıyla öne çıkar; ancak kapsamı dardır.

- **Huffman ve Aritmetik Kodlama** olasılığa dayanır; Aritmetik, Huffman'ın gelişmiş ve daha hassas halidir.

- **LZW** ise karakter gruplarını (stringleri) tanıyarak farklı bir sıkıştırma stratejisi uygular.

- **Ortak Nokta:** Dördü de veriden **hiçbir bilgi kaybetmez** (Kayıpsız).

---